

A Project Report

on

Dungeon Explorer

Submitted in partial fulfillment of the requirement of

Project – VI (BIT356CO)

Bachelor in Information Technology

Submitted to



Purbanchal University

Biratnagar, Nepal

Submitted by

Alisha Shahi (360801)

Bhavishek Lama (360805)

Riya Phuyal (360826)

KANTIPUR CITY COLLEGE

Putalisadak, Kathmandu

October, 2026

A Project Report

on

Dungeon Explorer

Submitted in partial fulfillment of the requirement of

Project – VI (BIT356CO)

Bachelor in Information Technology

Submitted To

Purbanchal University

Biratnagar, Nepal

Submitted By

Alisha Shahi (360801)

Bhavishek Lama (360805)

Riya Phuyal (360826)

Project Supervisor

Er Maryada Karki

Santosh Sah

KANTIPUR CITY COLLEGE

Putalisadak, Kathmandu

October, 2026

Abstract

Dungeon Explorer is a 2D tile-based action-adventure game developed using Python, designed to provide an engaging and dynamic gaming experience through AI-based procedural map generation. Players create or select an explorer profile explore dynamically generated dungeons, collect resources, find key-and-door objectives, fight monsters, and manage health, stamina, and inventory items. To improve replayability, the game generates a new dungeon for each run using a hybrid procedural generation pipeline. A genetic algorithm produces candidate layouts, cellular automata smooth terrain, breadth-first search repairs disconnected areas and measures reachability, and A* pathfinding validates routes for entities and objectives. The game also incorporates profile persistence through JSON files and uses recent player performance to adjust future monster limits through an explainable rule-based system. This project demonstrates the integration of game development, procedural content generation, pathfinding, artificial intelligence, adaptive difficulty, user interface design, and local data persistence within a single-player desktop application

Acknowledgement

We would like to express our heartfelt gratitude to everyone who has supported and encouraged us throughout the completion of this project.

First and foremost, our sincere thanks go to Purbanchal University for introducing such a practical and insightful course, which has provided us with the platform to explore and enhance our technical and academic capabilities.

We are deeply grateful to the entire team of Kantipur City College (KCC) for providing us with this valuable opportunity. We would especially like to extend our appreciation to the Principal of KCC, whose support and leadership have been instrumental in fostering an environment of learning and growth.

A special note of thanks goes to our respected supervisors, Santosh Sah and Maryada Karki, whose consistent encouragement, guidance and rich knowledge helped us throughout the course of this project. Their mentorship has been invaluable, and we could not have asked for better mentors.

We also acknowledge the contributions of each team member and our classmates, who shared suggestions, advice and experiences that greatly enriched this project.

Finally, we extend our warmest thanks to our friends and family who supported us both directly and indirectly. Your encouragement and help made this journey smoother and more meaningful.

Alisha Shahi (360801)

Bhavishek Lama (360805)

Riya Phuyal (360826)

Declaration

We declare that this project report titled “Dungeon Explorer”, submitted in partial fulfillment of the Bachelor of Information Technology, is a record of original work carried out by us under the supervision of Santosh Sah and Maryada Karki, and has not formed the basis for the award of any other degree or Diploma, in this or any other institution or University. In keeping with the ethical practice of reporting scientific information, due acknowledgments have been made wherever the findings of others have been cited.

Alisha Shahi (360801)

Bhavishek Lama (360805)

Riya Phuyal (360826)

Supervisor's Approval

This is to certify that the project entitled “Dungeon Explorer”, undertaken and demonstrated by Alisha Shahi, Bhavishek Lama and Riya Phuyal, has been successfully completed under our supervision as a partial fulfillment of the requirements of the degree of Bachelor of Information Technology, sixth semester, under Purbanchal University. We, henceforth, approve this project to be awarded the certificate by the concerned authority. During supervision, we found the students hardworking, skilled and ready to undertake any professional work related to this field in the future.

Mr. Santosh Sah

Project Supervisor

Department of IT

Er. Maryada Karki

Project Supervisor

Department of IT

Certificate From Department

Following the Supervisor's Approval and Examiner's Acceptance, the project entitled "Dungeon Explorer", submitted by Alisha Shahi, Bhavishek Lama and Riya Phuyal as a partial fulfillment of the requirements for the degree of BIT VI, under Purbanchal University, has been officially awarded by this certificate. We wish the students all the best for their future endeavors.

Mr. Saroj Pandey

Head of Department

Department of IT

Kantipur City College

Mr. Rubim Shrestha

Project Coordinator

Department of IT

Kantipur City College

Table of Contents

Abstract	iii
Acknowledgement	iv
Declaration	v
Supervisor's Approval	vi
Certificate From Department	vii
List of Figures	xi
List of Tables	xii
List of Abbreviations	xiii
1 Introduction.....	1
1.1 Overview.....	1
1.2 Problem Statement.....	1
1.3 Objectives	2
1.4 Features	2
1.5 Significance.....	3
1.6 Scope and Limitations.....	4
1.7 Documentation Organization.....	5
2 Literature Review.....	6
2.1 F.E.A.R. (2005).....	6
2.2 Pac-Man (1980)	7
2.3 Left 4 Dead	7
2.4 Terraria (2011).....	8
3 Methodology	9
3.1 Software Development Life Cycle.....	9
3.1.1 Iteration 1: Core Engine Development	9
3.1.2 Iteration 2: Procedural Dungeon Generation	10
3.1.3 Iteration 3: Monster AI and Combat.....	10
3.1.4 Iteration 4: Items, Gameplay and Player Progression.....	11
3.1.5 Iteration 5: Adaptive Difficulty and Player Analysis	12
3.1.6 Iteration 6: Integration and Final System Development	12

3.2	Technologies and Tools Used.....	13
3.2.1	Programming Language.....	13
3.2.2	Game Library	13
3.2.3	Data Storage.....	13
3.2.4	Development Environment and Version Control	14
3.3	AI and Data Mining Techniques Implemented.....	14
3.4	Assignment of Roles and Responsibilities.....	15
4	System Analysis.....	16
4.1	Requirement Analysis.....	16
4.1.1	Requirement Gathering.....	16
4.1.2	Functional Requirements	17
4.2	Feasibility Study	18
4.2.1	Technical Feasibility.....	18
4.2.2	Operational Feasibility.....	18
4.2.3	Economic Feasibility	18
4.2.4	Schedule Feasibility	19
5	System Design and Implementation	20
5.1	System Architecture.....	20
5.1.1	Presentation Layer	20
5.1.2	Gameplay Layer.....	20
5.1.3	Rendering Layer.....	21
5.1.4	AI and Procedural Generation Layer	21
5.1.5	Data and Persistence Layer.....	21
5.2	Procedure Oriented Diagrams.....	22
5.2.1	Data Flow Diagram — Level 0.....	22
5.2.2	Data Flow Diagram — Level 1.....	23
5.2.3	Use Case Diagram.....	24
6	System Development and Implementation	25
6.1	Programming Platform.....	25
6.2	Hardware and Software Requirements	25
6.2.1	Hardware Requirements.....	25
6.2.2	Software Requirements	26

7	Testing and Debugging	27
7.1	Testing Approach.....	27
7.2	Tools Used in Testing	27
7.3	Test Cases	28
7.3.1	Profile and Session Module	28
7.3.2	Procedural Map Generation and AI Modules	28
7.3.3	Gameplay, Combat and Collision.....	29
7.3.4	Monster AI	29
7.3.5	Items, Progression and Persistence Module.....	30
8	Conclusion and Future Work	31
	References.....	32

List of Figures

Figure 4-1:Gantt chart.....	19
Figure 5-1 System Architecture of Dungeon Explorer.....	20
Figure 5-2: Data Flow Diagram — Level 0 (Context Diagram)	22
Figure 5-3 Data Flow Diagram — Level 1	23
Figure 5-4 : Use Case Diagram.....	24

List of Tables

Table 3-1: Core Engine Development	9
Table 3-2: Procedural Dungeon Generation	10
Table 3-3: Monster AI and Combat	10
Table 3-4: Items, Gameplay and Player Progression.....	11
Table 3-5: Adaptive Difficulty and Player Analysis.....	12
Table 3-6: :Integration and Final System Development	12
Table 3-7: AI and Data Mining Techniques Used	14
Table 3-8: Assignment of Roles and Responsibilities	15
Table 4-1: Requirement Gathering	16
Table 4-2: Functional Requirements.....	17
Table 7-1: Tools Used in Testing.....	27
Table 7-2: Test Cases - Profile and Session Module	28
Table 7-3: Test Cases - Procedural Map Generation and AI Modules	28
Table 7-4: Test Cases - Gameplay, Combat and Collision.....	29
Table 7-5: Test Cases – Monster AI	29
Table 7-6: Test Cases - Items, Progression and Persistence Module	30

List of Abbreviations

AI	:	Artificial Intelligence
A*	:	A-Star (admissible heuristic pathfinding)
BFS	:	Breadth-First Search
CA	:	Cellular Automata
CSV	:	Comma-Separated Values
DFD	:	Data Flow Diagram
DTD	:	Document Type Definition
ER	:	Entity Relationship
FIFO	:	First In, First Out
FPS	:	Frames Per Second
GA	:	Genetic Algorithm
GUI	:	Graphical User Interface
HUD	:	Head-Up Display
IDE	:	Integrated Development Environment
JSON	:	JavaScript Object Notation
LOD	:	Level of Detail
NPC	:	Non-Player Character
PC	:	Personal Computer
RAM	:	Random Access Memory
SDLC	:	Software Development Life Cycle
SPA	:	Single-Page Application
UI	:	User Interface
XP	:	Experience Points

1 Introduction

1.1 Overview

Dungeon Explorer is a 2D procedural action-adventure game developed in Python using the Pygame library. Players create or select an explorer profile, and enter a randomly generated dungeon filled with monsters, treasures, keys, potions, traps, water, lava, and locked objectives.

The game uses a hybrid map-generation system that combines genetic algorithms, cellular automata, breadth-first search, and A* pathfinding. These techniques create different dungeon layouts while ensuring that important areas remain accessible. Players explore the environment, collect items, fight monsters using projectiles, manage health and stamina, and unlock doors by finding the required keys.

Dungeon Explorer also includes inventory management, character progression, persistent player profiles, scheduled monster spawning, and adaptive difficulty. Recent gameplay results influence the number of monsters generated in future runs.

1.2 Problem Statement

Most two-dimensional dungeon games rely on maps that were designed by hand or assembled from pre-built rooms. This creates a number of practical problems, which the project set out to address:

- **Repetitive gameplay:** Once a player has memorized a fixed level, replaying it adds nothing. Large hand-authored worlds are also extremely expensive to produce, so most games of this scale reuse a small number of levels.
- **Poor replay value from randomness alone:** Simply filling a grid with random tiles does not produce a good level. The result is usually full of unreachable regions, single-tile artefacts and awkward bottlenecks that make the level frustrating rather than interesting.
- **Difficulty does not adapt:** Fixed maps offer the same challenge to every player, whether they are playing for the first time or have completed the level fifty times.
- **Behavioral data is not used:** Player actions are normally discarded once a session ends, so nothing is learned about how people actually play, and no improvement is made to later content.

1.3 Objectives

The objectives of the project are:

- To develop a playable 2D tile-based dungeon game in Python using Pygame, featuring procedurally generated and fully playable cave-like levels using AI techniques.
- To implement an adaptive gameplay system that analyzes player behavior to dynamically adjust monster density and difficulty.

1.4 Features

Dungeon Explorer implements the following features:

- **Player Profile Management:** The system allows players to create, switch, and manage multiple local profiles. Each profile maintains its own progress, inventory, experience, and gameplay history.
- **Procedural Dungeon Generation:** A new 81×81 dungeon is automatically generated whenever a new gameplay run begins, providing different layouts, item placements, and monster positions for each gameplay session.
- **Guaranteed Level Playability:** The generated dungeon is validated using Breadth-First Search (BFS) to ensure that the areas are connected and accessible from the player's starting position.
- **Player Movement and Collision Detection:** The player can move through the dungeon using keyboard controls with tile-based collision detection to prevent movement through walls and other obstacles.
- **Sprinting and Stamina Management:** Players can sprint to move faster, but sprinting consumes stamina. Stamina automatically regenerates when sprinting is stopped.
- **Aiming and Ranged Combat:** The player can aim using the mouse and attack enemies with projectiles. Directional shooting animations and a controlled firing rate provide consistent combat behavior.

- **Monster Artificial Intelligence:** Monsters use a rule based, FSM-like behavior to manage actions such as becoming active, chasing the player, and attacking.
- **A* Pathfinding:** Monsters use A* pathfinding to navigate through the dungeon and find suitable routes toward the player while avoiding obstacles.
- **Environmental Hazards:** The dungeon contains hazards such as lava, traps, and water that affect the movement or health of both player and monster.
- **Fog of War:** Areas outside the player's visibility range remain hidden, creating an exploration challenge. Certain items can improve or remove the visibility limitation.
- **Items, Chests, and Keys:** The game includes health and stamina items, keys, treasure chests, and special utility items that provide additional abilities or advantages.
- **Inventory System:** Players can access an inventory during gameplay to view and use available consumable items. A separate inventory view is also available through the menu.
- **Experience and Leveling System:** Players gain experience by defeating monsters. Accumulated experience allows the player to level up, with increasing experience requirements for subsequent levels.
- **Player Statistics:** A statistics screen displays information such as player level, experience, games played, completed games, monster kills, item usage, and recent gameplay results.

1.5 Significance

- The project demonstrates the practical use of Artificial Intelligence algorithms in 2D game development.
- It provides dynamic and replayable gameplay by generating a different dungeon layout, item placement, and monster arrangement for each new gameplay run.

- It reduces the need for manual designing every dungeon by using procedural map generation and optimization techniques.
- The project helps demonstrate how data mining and player behavior analysis can be used to improve future gameplay and map generation.
- It provides practical learning experience in Python, AI algorithms, procedural generation, and game development for students.

1.6 Scope and Limitations

Scope

The project is a single-player desktop game designed to run on Windows, Linux, and macOS. It generates, validates, and runs procedurally generated dungeon levels. Monster behavior uses rule-based, FSM-like logic combined with A* pathfinding. Dungeon selection and monster limits can be adapted using recent gameplay history. The project also includes a local JSON persistence layer that stores player profiles, inventory, experience, monster kills, item usage, and recent game outcomes.

Limitations

The following features were considered but could not be implemented within the time available:

- The game is single-player and does not support multiplayer networking.
- The player behavior data is limited to a small number of recent gameplay sessions, so the data mining results are limited.
- The Genetic Algorithm uses a fixed population of eight candidates and limited generations (four) to reduce map-generation time.
- The map optimization focuses mainly on connectivity and exploration space and does not consider all aspects of game difficulty.
- Monster behavior uses predefined rules and does not include individual learning, reinforcement learning, or advanced group coordination.
- Very large dungeon maps may affect performance.
- The monster system is FSM-like but does not use a formal finite-state-machine class.

1.7 Documentation Organization

This report is organized into eight chapters. Chapter 1 introduces the project and states its objectives, features, scope and limitations. Chapter 2 reviews the commercial and academic games that use comparable techniques and identifies what the proposed system borrows from each. Chapter 3 describes the development methodology, the technologies used, the AI and data-mining techniques implemented, and the division of work among the team members. Chapter 4 presents the requirement analysis, the functional requirements and the feasibility study, together with the project schedule. Chapter 5 is the core of the report: it describes the system architecture, the overall game flow, the internal flow of the application and, in detail, the AI subsystem and the monster artificial intelligence, supported by data flow diagrams, a use case diagram and an entity-relationship model. Chapter 6 covers system development and implementation, including the hardware and software environment. Chapter 7 presents the testing approach and the test cases for each module. Finally, Chapter 8 concludes the report and identifies future improvements.

2 Literature Review

Artificial Intelligence and procedural generation have become important techniques in modern games for creating intelligent enemies, dynamic environments, and replayable gameplay. Different games use AI algorithms and procedural techniques to control enemy behavior, generate game worlds, and adapt gameplay according to player actions. This review examines existing games related to the proposed Dungeon Explorer system, highlighting their advantages and limitations and identifying the concepts adopted in the proposed project.

2.1 F.E.A.R. (2005)

F.E.A.R. is a first-person action game known for its intelligent enemy behavior. Its enemies use a Finite State Machine (FSM), Goal-Oriented Action Planning (GOAP), and A* pathfinding to respond to the player's actions. Enemies can patrol, search, attack, take cover, and move strategically through the environment.

Pros:

- Provides realistic and responsive enemy behavior.
- Uses FSM to organize different enemy states.
- Uses A* pathfinding for effective navigation.
- Demonstrates how relatively simple AI techniques can create convincing enemy behavior.

Cons:

- GOAP adds complexity to the AI system.
- Designed for a commercial 3D game and therefore is more complex than required for a small 2D project.
- Its advanced enemy behavior requires considerable development resources.

Relevance to Proposed System:

F.E.A.R. provides a reference for monster AI in Dungeon Explorer. The proposed system uses FSM and A* pathfinding to allow monsters to detect, chase, and attack the player while navigating around obstacles.

2.2 Pac-Man (1980)

Pac-Man is a classic arcade game that demonstrates simple but effective artificial intelligence. Ghosts use different movements and target behaviors while sharing basic behavioral states. This allows different enemies to behave differently without requiring highly complex algorithms.

Pros:

- Demonstrates simple and understandable game AI.
- Different enemy behaviors create variety in gameplay.
- Use a tile-based environment suitable for pathfinding.
- Efficient and suitable for resource-limited systems.

Cons:

- Enemy behavior is relatively simple.
- Does not provide complex adaptive difficulty.
- The fixed maze structure limits environmental variation.

Relevance to Proposed System:

Pac-Man demonstrates that different enemy behaviors can be created by modifying parameters within a common AI structure. Dungeon Explorer follows a similar approach by using a common FSM for its monster types while changing their movement and attack characteristics.

2.3 Left 4 Dead

Left 4 Dead is a cooperative survival game that introduced the “AI Director”, which monitors player performance and dynamically controls gameplay elements such as enemy spawning and attack intensity. The system aims to maintain an appropriate level of challenge based on the player's current situation.

Pros:

- Provides dynamic difficulty adjustment.
- Responds to player behavior and performance.
- Improves replayability by changing gameplay conditions.
- Combines local enemy AI with global gameplay management.

Cons:

- The AI Director is complex to implement.

- Designed for multiplayer gameplay, while Dungeon Explorer focuses on single-player gameplay.
- Requires continuous monitoring of several gameplay parameters.

Relevance to Proposed System:

Left 4 Dead provides the main inspiration for the adaptive difficulty feature of Dungeon Explorer. The proposed system records player behavior and uses rule base association to adjust monster density in subsequent runs according to the player's demonstrated performance.

2.4 Terraria (2011)

Terraria is a 2D sandbox adventure game that uses procedural generation to create unique worlds containing caves, terrain, structures, resources, and enemies. Its procedural approach provides high replayability by generating different environments for different game sessions.

Pros:

- Provides high replayability through procedural world generation.
- Generates natural-looking caves and environments.
- Uses rule-based techniques for placing structures and resources.
- Demonstrates the effectiveness of procedural generation in a 2D game.

Cons:

- The generated world is much larger and more complex than the dungeon required for this project.
- Most gameplay difficulty does not adapt directly to individual player performance.
- Its world-generation system is highly complex for a small-scale project.

Relevance to Proposed System:

Terraria is particularly relevant to the procedural dungeon generation component of Dungeon Explorer. The proposed system similarly uses Cellular Automata to create cave-like environments and connectivity checking to ensure that the generated dungeon remains playable.

3 Methodology

3.1 Software Development Life Cycle

This project follows the Agile Model to ensure the systematic and flexible development of the Dungeon Explorer game. The system is developed through multiple iterations, where each iteration delivers a functional version of the game with additional features and improvements. Continuous testing and refinement are performed throughout the development process to ensure that the gameplay, AI components, procedural generation, and adaptive difficulty work correctly.

3.1.1 Iteration 1: Core Engine Development

Table 3-1: Core Engine Development

Phase	Details
Requirement Gathering & Analysis	Identified the basic requirements of the game, including player movement, dungeon environment, collision detection, game objectives, and basic enemy interaction.
System Design	Designed the basic game structure, tile-based map, player and entity classes, screen management, and game loop.
Implementation	Implemented the Pygame environment, player movement, tile-based collision, basic dungeon map, game screens, and core game loop.
Testing	Tested player movement, collision detection, screen transitions, and basic gameplay to identify and fix initial errors.

3.1.2 Iteration 2: Procedural Dungeon Generation

Table 3-2: Procedural Dungeon Generation

Phase	Details
Requirement Gathering & Analysis	Identified the need for unique and replayable dungeon layouts instead of fixed maps.
System Design	Designed the procedural generation pipeline using Genetic Algorithm, Cellular Automata, BFS connectivity checking, and fitness evaluation.
Implementation	Implemented the 81×81 dungeon generation system, cave carving using Cellular Automata, candidate evaluation using a Genetic Algorithm, and BFS-based connectivity validation and repair.
Testing	Generated multiple dungeon layouts and tested their connectivity, structure, playability, and variation.

3.1.3 Iteration 3: Monster AI and Combat

Table 3-3: Monster AI and Combat

Phase	Details
Requirement Gathering & Analysis	Identified the need for intelligent enemies capable of detecting, chasing, and attacking the player.
System Design	Designed the monster with rule-based FSM-like behavior with states such as Dormant, Wake, Chase, and Attack. A* pathfinding was selected for obstacle-aware navigation.

Implementation	Implemented different monster types, FSM-based behavior, A* navigation, player aiming, projectile attacks, enemy attacks, and combat interactions.
Testing	Tested monster state transitions, pathfinding around obstacles, attack behavior, collision, and combat interactions.

3.1.4 Iteration 4: Items, Gameplay and Player Progression

Table 3-4: Items, Gameplay and Player Progression

Phase	Details
Requirement Gathering & Analysis	Identified additional gameplay requirements including items, inventory, keys, treasure chests, environmental hazards, fog of war, experience, and player progression.
System Design	Designed the inventory, item management, experience and levelling system, environmental hazards, fog of war, and game progression mechanisms.
Implementation	Implemented health and stamina items, Door Key, treasure chests, Bow, Light, inventory system, sprinting, environmental hazards, fog of war, experience, and levelling.
Testing	Tested item collection and usage, inventory operations, key and chest interactions, hazard effects, experience calculation, and level progression.

3.1.5 Iteration 5: Adaptive Difficulty and Player Analysis

Table 3-5: Adaptive Difficulty and Player Analysis

Phase	Details
Requirement Gathering & Analysis	Identified the need for the game to adjust difficulty according to the player's actual performance rather than using a fixed difficulty level.
System Design	Designed a player-history system to record kills, item usage, level completion, coins, and recent gameplay results. Association Rule analysis was selected to identify patterns in player behavior.
Implementation	Implemented gameplay history recording, Association Rule analysis, adaptive monster limit, profile statistics, and difficulty adjustment based on recent player performance.
Testing	Tested player-history recording, rule generation, difficulty adjustments, and whether monster limit changed appropriately according to different player performance patterns.

3.1.6 Iteration 6: Integration and Final System Development

Table 3-6: Integration and Final System Development

Phase	Details
Requirement Gathering & Analysis	Identified the need to integrate all game components into a stable and complete system.
System Design	Planned the integration of gameplay, procedural generation, monster AI, combat, inventory, player profiles, persistence, statistics, and adaptive difficulty.

Implementation	Integrated all modules into the complete Dungeon Explorer system and implemented profile management, data persistence, run rollback, statistics, and final game flow.
Testing	Conducted end-to-end testing of complete gameplay sessions, procedural generation, AI behavior, combat, inventory, progression, adaptive difficulty, and data persistence. Bugs were fixed and the final system was refined for stable operation

3.2 Technologies and Tools Used

3.2.1 Programming Language

Python is the primary programming language used to develop the Dungeon Explorer system. Python was selected because of its simplicity, flexibility, and extensive standard library support. Libraries such as `collections.deque`, `heapq`, `random`, `secrets`, `json`, and `data classes` support algorithms including BFS, A* pathfinding, random generation, data persistence, and game-world modelling. The project also uses Pygame for game development and graphical rendering. The major AI, procedural generation, and data-mining algorithms are implemented directly in Python rather than relying on pre-built AI or data-mining frameworks.

3.2.2 Game Library

Pygame CE (pygame-ce 2.5.8) is used as the primary game development library. It provides the functionality required for window management, event handling, keyboard and mouse input, sprite rendering, collision detection, image processing, transformations, and font handling. Pygame is mainly used for the presentation and input layers, while the core game logic, procedural generation, and AI algorithms are implemented separately to maintain a modular system.

3.2.3 Data Storage

The system uses JSON for local data storage. Player profiles, progress, inventory, statistics, and gameplay history are stored in a JSON file using Python's built-in `json` module. JSON was selected because the application is a single player, locally

running system and does not require a separate database server. It is lightweight, easy to manage, and human-readable, making it suitable for storing and inspecting game data during development and testing.

3.2.4 Development Environment and Version Control

The project is developed using Visual Studio Code (VS Code) with the Python extension. A dedicated Python virtual environment is used to manage the project dependencies. Git and GitHub are used for version control, allowing changes to be tracked throughout development and providing a reliable way to manage different development stages and maintain previous versions of the project.

3.3 AI and Data Mining Techniques Implemented

The Dungeon Explorer system implements several Artificial Intelligence (AI) and Data Mining techniques for procedural dungeon generation, enemy navigation, map evaluation, and adaptive difficulty. These techniques work together to generate playable dungeon layouts and adjust gameplay according to player behavior. Below table presents the techniques, their source modules, and their roles in the system.

Table 3-7: AI and Data Mining Techniques Used

Technique	Source module	Role in the system
Genetic Algorithm	genetic_algorithm.py	Evolves candidate dungeon layouts through multiple generations to produce improved map structures.
Cellular Automata	cellular_automata.py	Converts raw random patterns into natural-looking cave structures using neighborhood-based rules.
Breadth-First Search	bfs.py	Finds reachable cells, calculates distances, evaluates connectivity, and repairs disconnected walkable regions.
A* Pathfinding	astar.py	Finds walkable paths through the dungeon and enables monsters to navigate around obstacles toward the player.
Profile-based Decisions	decision_tree.py	Scores and selects candidate maps using reachability, openness, level, kills, and item usage.
Association Rule Analysis	decision_tree.py	Calculates recent-history confidence values and uses them to adjust the next run's monster limit.

3.4 Assignment of Roles and Responsibilities

Table 3-8: Assignment of Roles and Responsibilities

Group member	Technical responsibilities	Documentation responsibilities
Alisha Shahi	• Game loop and ScreenManager architecture • Player and Monster classes • Collision detection and combat system • A Pathfinding for monster navigation* • Breadth-First Search (BFS) for connectivity validation • Level completion and death-state handling	• Chapter 1: Introduction • Chapter 2: Literature Review • Chapter 4: System Analysis
Bhavishek Lama	• Procedural dungeon generation • Genetic Algorithm for dungeon optimization • Cellular Automata for cave generation • Decision Tree-based fitness evaluation • Map-generation integration and testing	• Chapter 5: System Design and AI subsystem • Chapter 6: Development and Implementation
Riya Phuyal	• User interface and screen modules • Setup, menu, inventory, profile, and pause screens • Button and TextInput components • WorldRenderer and sprite handling • Association Rule Mining for adaptive difficulty • Profile and gameplay-data persistence	• Chapter 3: Methodology • Chapter 7: Testing and Debugging

4 System Analysis

4.1 Requirement Analysis

Requirement analysis is the process of identifying and documenting the functional and non-functional needs of a system. It helps define what the system should do and how it should perform.

In this project, requirement analysis identifies the needs of the Dungeon Explorer game developed using Python and Pygame CE. The analysis focuses on player movement, combat, items, enemy behavior, user interface, profiles, and game progress. It also defines the requirements for AI-based procedural generation using Genetic Algorithm, Cellular Automata, BFS, A* Pathfinding, rule-based Decision, and Association Rule Analysis. A key requirement is that every generated dungeon must be connected and playable.

4.1.1 Requirement Gathering

Requirement gathering is the process of identifying user needs and system expectations. In this project, requirements were gathered through team discussions, game analysis, prototype testing, and supervisor feedback, focusing on gameplay, AI-based generation, enemy behavior, and adaptive difficulty.

Table 4-1: Requirement Gathering

ID	Requirement
R1	The system shall allow the player to create and manage local game profiles.
R2	The system shall generate a new playable dungeon using procedural generation techniques.
R3	The system shall use Genetic Algorithm, Cellular Automata, BFS, A* Pathfinding, and profile-based decision techniques for dungeon generation, validation.
R4	The system shall check generated dungeons are connected and playable by repairing unreachable areas.
R5	The system shall provide player movement, combat, items, inventory, and enemy interaction.

R6	The system shall provide intelligent monster behaviour using FSM-like behaviour and A* Pathfinding.
R7	The system shall analyse recent player history using Association Rule to adapt monster density and difficulty.
R8	The system shall store player profiles, progress, inventory, statistics, and gameplay history using local JSON persistent.

4.1.2 Functional Requirements

Table 4-2: Functional Requirements

ID	Function	Cross Reference
F1	Create, select, and load a local player profile.	R1
F2	Generate a new dungeon layout for each new gameplay run.	R2, R3
F3	Evaluate dungeon layouts and select a suitable playable layout.	R3
F4	Validate dungeon connectivity using BFS and repair unreachable regions.	R4
F5	Move the player with collision detection, camera following, and fog of war.	R5
F6	Allow the player to attack monsters using projectiles and receive damage from enemies.	R5
F7	Manage items, chests, keys, inventory, and the locked exit door.	R5
F8	Control monster behavior using FSM-like behaviors and A* Pathfinding.	R6
F9	Record player actions and recent run outcomes for gameplay analysis.	R7
F10	Apply lightweight Association Rule Analysis to adjust monster density for subsequent levels.	R7
F11	Save and load player progress, inventory, statistics, and gameplay history using JSON.	R8

F12	Track experience, level progression, kills, and game statistics.	R8
-----	--	----

4.2 Feasibility Study

Feasibility study determines whether the proposed system can be developed successfully using the available technology, resources, budget, and time. For Dungeon Explorer, feasibility is evaluated under four main aspects: technical, operational, economic, and schedule feasibility.

4.2.1 Technical Feasibility

Technical feasibility assesses whether the required technologies and skills are available and suitable. For this project:

- Python and Pygame CE are used for game development.
- The project implements BFS, A*, genetic operations, cellular-automata smoothing, and profile-analysis rules directly in Python using standard libraries such as collections, heapq, and random.
- The team has the required skills in programming, AI, and game development.

The system is technically feasible as the required tools and expertise are available.

4.2.2 Operational Feasibility

Operational feasibility evaluates whether the system can be easily operated by its users. For this project:

- The game provides a simple menu and user interface.
- Basic keyboard and mouse controls are used for gameplay.
- The game operates locally without requiring an internet connection.

The system is operationally feasible.

4.2.3 Economic Feasibility

Economic feasibility evaluates whether the project can be developed at a reasonable cost. For this project:

- Python, Pygame CE, VS Code, Git, and GitHub are free to use.
- No special hardware or paid software is required.
- Development is carried out using existing computers.

The project is economically feasible with minimal development cost.

4.2.4 Schedule Feasibility

Schedule feasibility examines whether the project can be completed within the available time frame. For this project:

- Development is completed within the allocated project period.
- Major activities include requirement analysis, design, implementation, testing, and documentation.
- Tasks are divided among team members.

The project is schedule feasible and can be completed within the given time.

4.2.4.1 Gantt chart

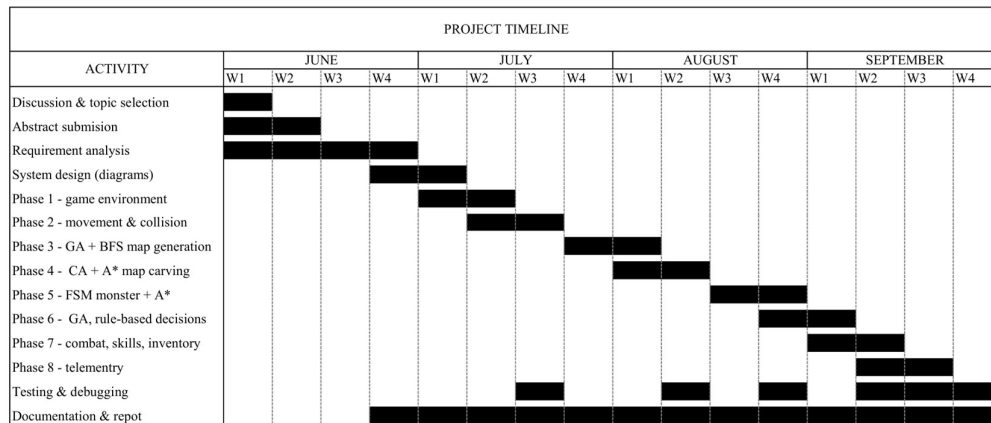


Figure 4-1: Gantt chart

5 System Design and Implementation

5.1 System Architecture

Dungeon Explorer follows a layered architecture that separates the user interface, gameplay, rendering, AI-based dungeon generation, and data persistence. This structure improves modularity and makes the system easier to develop and maintain.

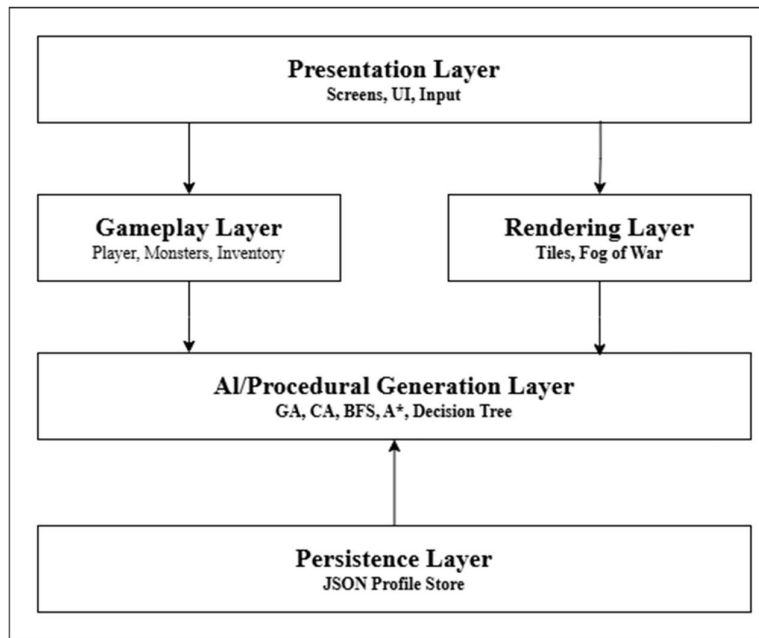


Figure 5-1 System Architecture of Dungeon Explorer

5.1.1 Presentation Layer

- Provides menus, game screens, inventory, profile, and pause interfaces.
- Handles keyboard and mouse input.
- Displays game information to the player.

5.1.2 Gameplay Layer

- Manages player movement, collision, combat, items, inventory, and game progression.
- Controls monster behavior using Finite State Machine ideology.
- Handles level completion, death, and game states.

5.1.3 Rendering Layer

- Renders dungeon tiles, players, monsters, items, effects, and environmental elements.
- Manages camera movement and fog of war.
- Uses Pygame CE for visual rendering.

5.1.4 AI and Procedural Generation Layer

This layer provides the intelligent features of the game:

- Genetic Algorithm (GA): evolves candidate dungeon layouts.
- Cellular Automata (CA): smooths local terrain patterns.
- BFS: checks dungeon connectivity and repairs unreachable areas.
- A* Pathfinding: find paths for monster navigation and assist with entity placement.
- Profile-based Decision evaluates generated layouts using gameplay and level-based criteria.
- Association Rule: lightweight association-rule analysis adjusts the monster limit for the next run.

5.1.5 Data and Persistence Layer

- Stores player profiles, inventory, statistics, progress, and gameplay history.
- Uses JSON for local data storage.
- Supports saving, loading, and rollback of game progress.

5.2 Procedure Oriented Diagrams

5.2.1 Data Flow Diagram — Level 0

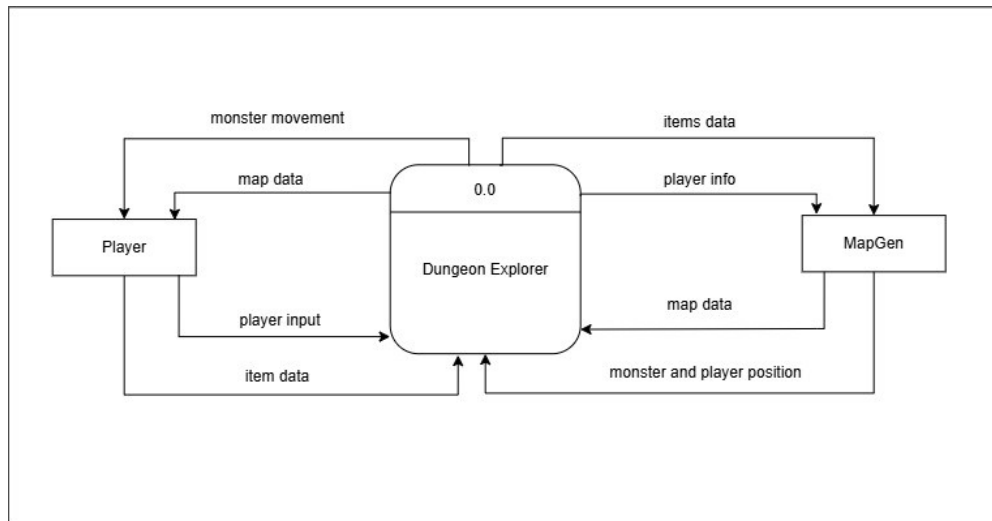


Figure 5-2: Data Flow Diagram — Level 0 (Context Diagram)

5.2.2 Data Flow Diagram — Level 1

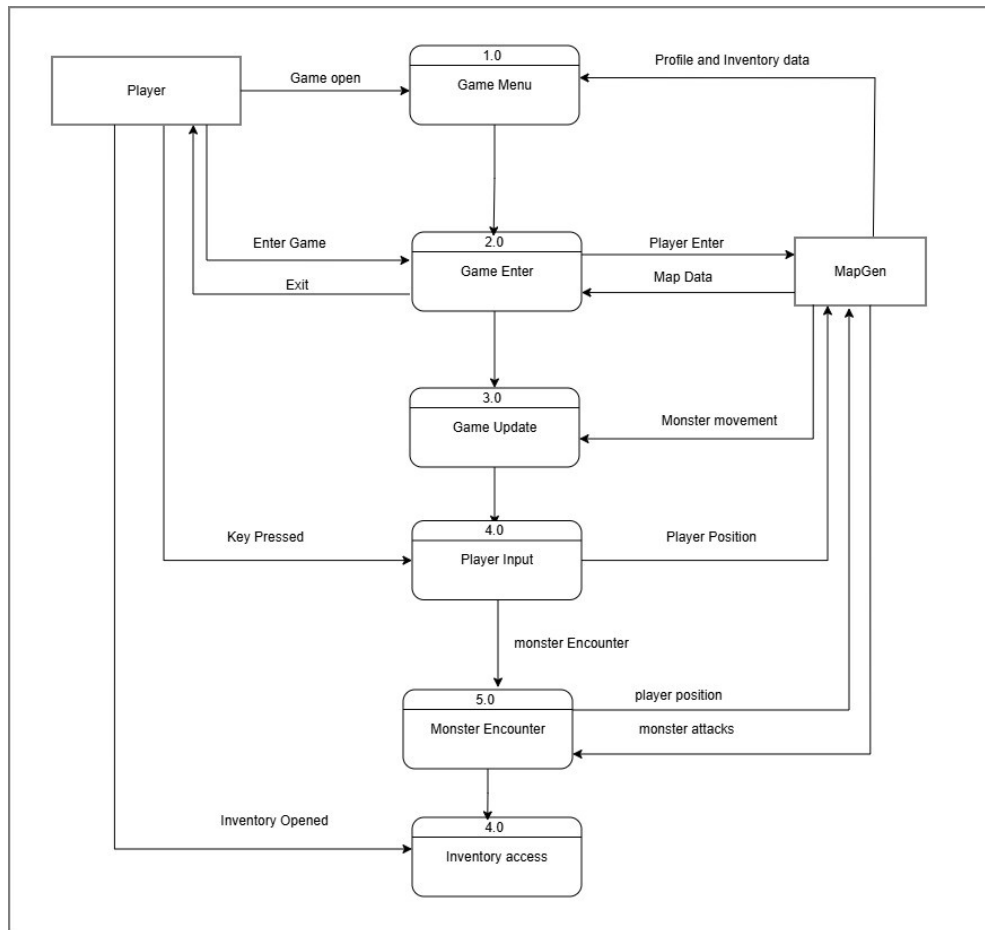


Figure 5-3 Data Flow Diagram — Level 1

5.2.3 Use Case Diagram

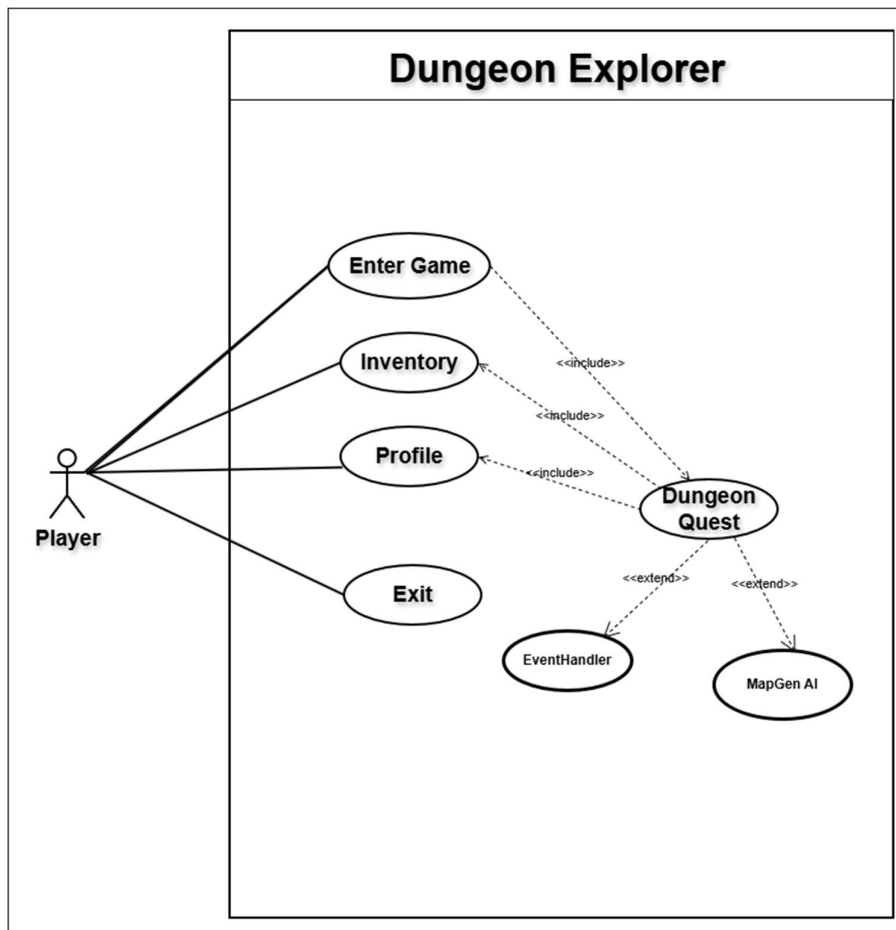


Figure 5-4 : Use Case Diagram

6 System Development and Implementation

6.1 Programming Platform

1. Main Language: Python 3

- Python is used as the main programming language for game development, gameplay logic, AI algorithms, data-mining techniques, and data persistence.

2. Game Library: Pygame CE 2.5.8

- Pygame CE is used for game development, including window management, keyboard and mouse input, game rendering, collision detection, sprites, images, and fonts.

3. Integrated Development Environment: Visual Studio Code

- Visual Studio Code with the Python extension is used for coding, debugging, testing, and project development.

4. Version Control: Git and GitHub

- Git is used to track changes and manage different versions of the project, while GitHub is used to host the project repository.

5. Data Format: JSON

- JSON is used for storing player profiles, game progress, inventory, statistics, and gameplay history. It is suitable because the game is single-player and operates locally without requiring a database server.

6. Asset Pipeline

- PNG-based sprites and tile images are used for the player, monsters, items, and dungeon environment. These assets are managed within the project and loaded during gameplay.

6.2 Hardware and Software Requirements

6.2.1 Hardware Requirements

Recommended Requirements

- Processor: Intel Core i5 or AMD Ryzen 5 or faster
- RAM: 8 GB
- Storage: 500 MB free space
- Display: 1920 × 1080 resolution

No dedicated graphics card or internet connection is required to run the game.

6.2.2 Software Requirements

- Operating System: Windows 10 or 11, Linux, or macOS
- Python: 3.10 or later
- Game Library: Pygame CE 2.5.8
- Integrated Development Environment: Visual Studio Code with the Python extension
- Version Control: Git and GitHub
- Environment Management: Python Virtual Environment (venv)

7 Testing and Debugging

Testing was planned and carried out through module checks, to verify that the game functions correctly, generated dungeons are playable, AI algorithms work as expected, and player progress is properly maintained. Testing included AI validation, gameplay testing, and manual testing.

7.1 Testing Approach

The system was tested at three levels:

- AI and Procedural Testing: Generated dungeons were tested for connectivity, valid player spawn, reachable key and door, and proper pathfinding.
- Gameplay Testing: Player movement, collision, combat, items, inventory, monster behavior, level completion, and death/rollback were tested.
- Data and Adaptive Testing: Player profiles, saved progress, gameplay history, and adaptive monster difficulty were tested using different player scenarios.

Failed test cases were identified, corrected, and tested again to verify the fixes.

7.2 Tools Used in Testing

Table 7-1: Tools Used in Testing

S.N.	Tool	Specification and purpose
1	Development hardware	Laptop with an AMD Ryzen 7 processor and 16 GB RAM, running Windows 11, used to run the game
2	Python 3	Testing AI algorithms and game logic
3	Pygame CE 2.5.8	Testing gameplay, input, rendering, and game behaviour
4	Visual Studio Code	Debugging and inspecting program execution
5	profiles.json inspection	Verifying saved profiles, progress, and gameplay history

7.3 Test Cases

The major functional and AI components of Dungeon Explorer were tested to verify their correctness and reliability.

7.3.1 Profile and Session Module

Table 7-2: Test Cases - Profile and Session Module

TC ID	Description	Input / Condition	Expected Output	Actual Output	Status
TC-P01	Create player profile	Enter a valid player name	New profile is created	Profile created successfully	Pass
TC-P02	Load existing profile	Select a saved profile	Profile data is loaded	Profile loaded successfully	Pass
TC-P03	Save and restore progress	Play a level and restart the game	Previous progress is restored	Progress restored correctly	Pass

7.3.2 Procedural Map Generation and AI Modules

Table 7-3: Test Cases - Procedural Map Generation and AI Modules

TC ID	Description	Input / Condition	Expected Output	Actual Output	Status
TC-A01	Generate dungeon	Start a new level	A valid 81×81 dungeon is generated	Dungeon generated successfully	Pass
TC-A02	Check dungeon connectivity	Generate multiple dungeons and apply BFS	All walkable regions are reachable	All tested dungeons were connected	Pass
TC-A03	Test A* pathfinding	Monster and player placed with obstacles between them	A valid path is found	Monster reached the player	Pass

TC-A04	Test adaptive difficulty	Provide different recent player histories	Monster limit changes according to the history	Monster count adjusted correctly	Pass
--------	--------------------------	---	--	----------------------------------	------

7.3.3 Gameplay, Combat and Collision

Table 7-4: Test Cases - Gameplay, Combat and Collision

TC ID	Description	Input / Condition	Expected Output	Actual Output	Status
TC-G01	Player movement	Press WASD or arrow keys	Player moves in the selected direction	Player moved correctly	Pass
TC-G02	Wall collision	Move player into a wall	Player cannot pass through the wall	Collision worked correctly	Pass
TC-G03	Player attack	Aim and fire at a monster	Monster receives damage	Monster was damaged successfully	Pass
TC-G04	Environmental hazard	Move player into lava or trap	Player receives the defined damage/effect	Damage/effect applied correctly	Pass

7.3.4 Monster AI

Table 7-5: Test Cases – Monster AI

TC ID	Description	Input / Condition	Expected Output	Actual Output	Status
TC-M01	Monster activation	Move player near a dormant monster	Monster changes to active state	Monster movement achieved	Pass

TC-M02	Monster chase	Player is detected by monster	Monster follows the player	Monster chased the player correctly	Pass
TC-M03	Monster attack	Player enters attack range	Monster attacks the player	Attack and damage worked correctly	Pass

7.3.5 Items, Progression and Persistence Module

Table 7-6: Test Cases - Items, Progression and Persistence Module

TC ID	Description	Input / Condition	Expected Output	Actual Output	Status
TC-I01	Item collection	Player collects an item	Item is added to inventory	Item added successfully	Pass
TC-I02	Door Key and level completion	Collect Door Key and open the locked door	Level is completed and progress is saved	Level completed and progress saved	Pass
TC-I03	Experience and level-up	Defeat monsters and gain required XP	Player level increases	Level increased correctly	Pass
TC-I04	Inventory persistence	Collect items and restart the game	Saved inventory is restored	Inventory restored correctly	Pass

8 Conclusion and Future Work

The Dungeon Explorer project successfully demonstrates the use of Artificial Intelligence and Data Mining techniques in a 2D tile-based dungeon game. The system generates different dungeon layouts, checks their connectivity, optimizes the maps, and adapts gameplay based on player behavior. The project also demonstrates practical use of Python, procedural generation, pathfinding, and game development concepts while providing a dynamic and replayable gaming experience.

References

Papp, T. (2020, April 23). *F.E.A.R.: shortest path for realistic AI – Tamás P.*

Papp. <https://www.lancaster.ac.uk/stor-i-student-sites/tamas-papp/2020/04/23/fear-shortest-path-for-challenging-ai/>

Making sure you're not a bot! (n.d.).

https://developer.valvesoftware.com/wiki/Left_4_Dead_Design_Theory

Sheen, T., & Sheen, T. (2026, April 12). *How Pac-Man Ghost AI Works the*

classic Chase algorithms. AI Ghost. <https://aighost.co.uk/how-pac-man-ghost-ai-works-the-classic-chase-algorithms/>

Pygame Front Page — pygame v2.6.0 documentation. (n.d.).

<https://www.pygame.org/docs/>